

Proyecto Final – Inteligencia Artificial

Juan Andrés Loncharich, 14534, [JuanLoncharich](#)

Diego Alejandro Forni, 14329, [DiegoForni](#)

Repositorio oficial:

https://github.com/diegoforni/llm_defender

Proyecto: Delfín

Agente LLM defensor para redes virtualizadas

Descripción

Laboratorio mínimo para evaluar si un **Agente LLM defensor** detecta y **previene/frena** intentos de **fuerza bruta SSH** contra un servidor con **PostgreSQL**. El tráfico “malicioso” es **manual y controlado** desde un cliente (descubrimiento previo + intento de fuerza bruta).

Aclaración clave: el **Defender solo genera prompts** (plan de defensa); [OpenCode](#) ejecuta esos prompts en el servidor destino.

Objetivo principal

Medir, en condiciones reproducibles, la capacidad del sistema para:

1. **Prevenir** el ataque SSH **antes** de que ocurra (tras detectar el escaneo).
2. **Frenar/bloquear** el ataque si ya inició.
3. Registrar **mecanismo aplicado**, **latencias** y **consistencia**.

Alcance (y no-alcance)

- **Sí:** un **cliente** (atacante simulado) y un **servidor** con **PostgreSQL + SSH, SLIPS** para señales y un **Defender LLM** que transforma alertas en prompt; **OpenCode** en el servidor **ejecuta** los prompts generados.

- **Ataques de prueba (solo lab):**
 - a) **Descubrimiento** desde el cliente (p. ej., escaneo para localizar el servidor).
 - b) **Fuerza bruta SSH** controlada contra una **cuenta de prueba** sin privilegios.
- **No:** explotación de vulnerabilidades, movimiento lateral, exfiltración, ni guías operativas fuera del lab.

Justificación

Los sistemas tradicionales de detección de intrusiones generan alertas ante comportamientos sospechosos, pero dependen de la intervención humana para responder, lo que introduce demoras críticas frente a ataques rápidos como la fuerza bruta SSH. Este proyecto propone un enfoque alternativo: un **agente LLM defensor** que interpreta las alertas de SLIPS y genera **comandos adaptables de defensa**, estructurados como planes ejecutables por OpenCode dentro de un entorno controlado y trazable, sin intervención humana directa.

El objetivo es evaluar si un modelo de lenguaje puede desempeñar un rol defensivo inteligente, capaz de **anticipar o frenar ataques en tiempo casi real** mediante la creación dinámica de estrategias ajustadas al contexto. Esta integración de IA y ciberseguridad busca demostrar la viabilidad de agentes autónomos que no solo detecten, sino que también generen respuestas adaptativas y reversibles, aportando evidencia experimental en un laboratorio reproducible y seguro.

Supuestos y restricciones

- Entorno **aislado** y bajo control.
- Defender y **SLIPS** corren en el **mismo contenedor**; [OpenCode](#) ejecuta en el **servidor**.
- Trazabilidad total: eventos → alerta → prompt → ejecución (OpenCode) → resultado.

Métricas y criterios

- **Prevención pre-ataque (%)**: corridas donde, tras detectar el escaneo, se aplica una defensa **antes** del inicio de fuerza bruta.
- **Bloqueo SSH (%)**: corridas donde la fuerza bruta **no** logra autenticarse.
- **Latencias (tiempo de demora hasta solución)**:

- **t_alert**: evento → alerta en Defender
- **t_decision**: alerta → prompt generado
- **t_exec**: recepción del prompt → ejecución por OpenCode → efecto aplicado

Protocolo de prueba (resumen)

1. **Escenario único**: “descubrimiento → fuerza bruta SSH (controlada)”.
2. **N corridas** reproducibles.
3. Recolección: timestamps, IPs, alertas, prompt, ejecución (OpenCode), mecanismos, latencias, resultado (prevented/blocked).
4. Reversión automática de endurecimientos temporales y limpieza del entorno.

Etapas

1) Infraestructura 3 días

Levanta dos subredes aisladas y enrutadas (A: 172.30.0.0/24 para el **Client**, B: 172.31.0.0/24 para el **Server** con PostgreSQL+SSH), un **Router** con IF-A/IF-B, un **Switch gestionado** con **SPAN** hacia el **Slips container** (donde corre **SLIPS + Defender LLM**), y verifica: reachability entre subredes (ICMP/TCP a 22 y 5432), sincronía NTP, rotación de logs en Server y Slips, políticas de reinicio y health checks básicos (servicio SSH vivo, PostgreSQL accesible local, SLIPS escuchando la interfaz espejo y Defender con endpoint de ingesta listo).

2) Código malicioso a ejecutar (nmap + brute force) 2 días

Desde el **Client** se ejecuta un **descubrimiento limitado** en la subred B (nmap) para ubicar el Server (solo dentro del lab) y validar exposición de 22/5432; luego, se generan **intentos controlados de autenticación SSH** contra una **cuenta de prueba sin privilegios**, con límites estrictos (p. ej., ≤30 intentos totales, ≤2 intentos/seg, ventana ≤2 min, lista de laboratorio corta, sin credenciales reales) y marcado de timestamps para latencias.

3) Alertas de SLIPS 2 días

SLIPS recibe el tráfico del **SPAN** y emite dos tipos de alertas, una para horizontal scan, y otra para SSH password guessing. Se tiene que verificar este funcionamiento.

4) Defender a partir de alertas 6 días

El **Defender** parsea la alerta, construye un **prompt defensivo** y produce un plan en JSON validado por esquema; las acciones posibles incluyen block_ip temporal (firewall del Server), rate_limit/delay_login en SSH y endurecimiento temporal de parámetros para la

cuenta de prueba; el prompt deja explícitos: IP origen, TTL del bloqueo, comandos declarativos de alto nivel y condiciones de revert, y **no ejecuta** nada por sí mismo: solo **genera el prompt** que pasará a ejecución.

5) Prueba de OpenCode 4 días

Antes del end-to-end, se valida la **ruta de ejecución** en el Server: se inyecta una **alerta sintética** al Defender, éste genera un prompt mínimo (p. ej., “bloquear IP X por 5 min y confirmar regla”), **OpenCode** lo ejecuta en el Server y el sistema recoge exit_code, stdout/stderr, tiempos (t_exec) y estado final (resultado de la ejecución).

6) Ejecución completa 6 días

Con todo enlazado, se realizan **corridas reproducibles**: (1) descubrimiento desde Client, (2) inicio de intentos controlados de SSH; se espera que el Defender, tras la alerta de scan, **prevenga** (bloqueo/limitación antes del brute force) o, en su defecto, **frene** durante el intento; por corrida se registra prevented_pre_attack(bool), ssh_blocked(bool), mecanismo aplicado, t_alert, t_decision, t_exec, y se consolida un resumen (tasas de prevención/bloqueo, percentiles de latencia, consistencia); al final se ejecuta **revert/cleanup** para dejar el lab en estado base.

7) Reporting 4 días

Compilación de resultados, generación de informes sobre los resultados del paso 6, junto con la presentación final del proyecto.

